99
- x
kodex
A coding CLI agent in ~200 lines of Python
Sangwhan Moon, sxm@
ROUND 99
READY
EXTEND
EXTEN

What is an LLM?
A neural network that predicts the **next token**.
predicted
The
cat
sat
on
the
mat
Autoregressive: predict one token, append, repeat. [1]
ROUND 99
READY

What is an LLM?

A neural network that predicts the **next token**.

predicted

The | cat | sat | on | the | mat

Autoregressive: predict one token, append, repeat. [1]

The Math

Next-token probability:

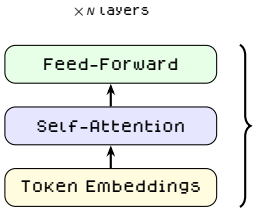
$$P(x_t \mid x_1, \ldots, x_{t-1};\ \theta)$$

Training minimizes cross-entropy:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t};\ \theta)$$

The Transformer

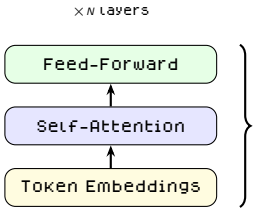
The architecture behind every modern LLM. [2]



Each token becomes a vector (embedding). Stack attention + FFN layers to build deep representations.

The Transformer

The architecture behind every modern LLM. [2]



Each token becomes a vector (embedding). Stack attention + FFN layers to build deep representations.

Self-Attention

Every token attends to every other token:

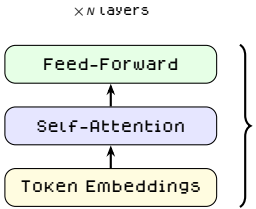
$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q, K, V = learned projections of input
- $\sqrt{d_k}$ scaling prevents gradient vanishing
- Cost: $O(n^2)$ in sequence length

This is why context windows are finite.

The Transformer

The architecture behind every modern LLM. [2]



Each token becomes a vector (embedding). Stack attention + FFN layers to build deep representations.

Self-Attention

Every token attends to every other token:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q, K, V = learned projections of input
- $\sqrt{d_k}$ scaling prevents gradient vanishing
- Cost: $O(n^2)$ in sequence length

This is why context windows are finite.

Why Transformers Won

Parallelizable (unlike RNNs), scale predictably, and attention lets the model learn **which tokens matter** for each prediction.

Tokens

LLMs see **tokens**, not characters.

"Unbelievably fast!" = 5 tokens

Un | believ | ably | _fast | !

~1 token per 4 chars in English. Code and math cost more. [3]

Tokens

LLMs see **tokens**, not characters.

"Unbelievably fast!" = 5 tokens

Un | believ | ably | _fast | !

~1 token per 4 chars in English. Code and math cost more. [3]

Context Window

Max tokens the model sees at once.

- GPT-4o: 128K
- Gemini 2.5: 1M
- Claude: 200K

Bounded by $O(n^2)$ attention cost. Prompt + history + tool results must fit. [4]

Tokens

LLMs see **tokens**, not characters.

"Unbelievably fast!" = 5 tokens

Un | believ | ably | _fast | !

~1 token per 4 chars in English. Code and math cost more. [3]

Context Window

Max tokens the model sees at once.

- GPT-4o: 128K
- Gemini 2.5: 1M
- Claude: 200K

Bounded by $O(n^2)$ attention cost. Prompt + history + tool results must fit. [4]

Why LLMs Are Stochastic

The model outputs a **probability distribution**, not a single answer. We **sample** from it:

$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Temperature T controls the distribution shape. Same prompt, different run = different sample = different answer.

Tokens

LLMs see **tokens**, not characters.

"Unbelievably fast!" = 5 tokens

Un | believ | ably | _fast | !

~1 token per 4 chars in English. Code and math cost more. [3]

Context Window

Max tokens the model sees at once.

- GPT-4o: 128K
- Gemini 2.5: 1M
- Claude: 200K

Bounded by $O(n^2)$ attention cost. Prompt + history + tool results must fit. [4]

Why LLMs Are Stochastic

The model outputs a **probability distribution**, not a single answer. We **sample** from it:

$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Temperature T controls the distribution shape. Same prompt, different run = different sample = different answer.

Sampling Controls

- **T=0**: always pick top token (deterministic)
- **T=1**: sample proportionally (creative)
- **Top-K**: only sample from K most likely
- **Top-P**: only sample from smallest set with cumulative prob $\geq$ P

How LLMs Are Built

Pre-training
internet-scale text
→
Mid-training
domain data, long ctx
→
Post-training
SFT + RLHF
→
Deployment
API, safety filters

How LLMs Are Built

Pre-training
internet-scale text
→
Mid-training
domain data, long ctx
→
Post-training
SFT + RLHF
→
Deployment
API, safety filters

Training stages

- **Pre-training**: Next-token on trillions of words
- **Mid-training**: Curated data (code, math, long docs)

Alignment

- **SFT**: Fine-tune on instruction/response pairs
- **RLHF**: Optimize for human preferences [5]

Base Model to Agent

A base model just predicts tokens. We add:

1. **SFT** – follow instructions
2. **Tool use** – emit structured calls
3. **System prompt** – role + rules
4. **Harness** – code that runs the loop

Base Model to Agent

A base model just predicts tokens. We add:

1. **SFT** – follow instructions
2. **Tool use** – emit structured calls
3. **System prompt** – role + rules
4. **Harness** – code that runs the loop

What is a Harness?

Code that wraps an LLM API:

- Manage **message history**
- Send **tool schemas**
- Execute **tool calls**
- Loop until **done**
- Enforce **safety**

Claude Code, Gemini CLI, Cursor, kodex.

SFT Teaches Turn-by-Turn Tool Use

SFT training data contains **full conversations** with tool calls baked in. The model learns the pattern:

SFT Teaches Turn-by-Turn Tool Use

SFT training data contains **full conversations** with tool calls baked in. The model learns the pattern:

Training sample

```
{"role": "user",
 "content": "What's in main.py?"}
{"role": "assistant",
 "tool_calls": [{"function":
   {"name": "read_file",
    "arguments": "{\"path\":\"main.py\"}"
 }}]}
{"role": "tool",
 "content": "import sys\ndef main():..."}
{"role": "assistant",
 "content": "main.py defines an entry
  point that..."}
```

SFT Teaches Turn-by-Turn Tool Use

SFT training data contains **full conversations** with tool calls baked in. The model learns the pattern:

Training sample

```
{"role": "user",
 "content": "What's in main.py?"}
{"role": "assistant",
 "tool_calls": [{"function":
   {"name": "read_file",
    "arguments": "{\"path\":\"main.py\"}"
 }}]}
{"role": "tool",
 "content": "import sys\ndef main():..."}
{"role": "assistant",
 "content": "main.py defines an entry
  point that..."}
```

What the model learns

1. When to **call a tool** vs. reply directly
2. How to **format** structured tool calls
3. How to **use tool results** in the answer
4. When to **chain** multiple tool calls

Thousands of these examples teach the model the agentic conversation pattern.

The Harness is the Security Boundary

The model can request **any tool call**. The harness decides what actually runs.

In production harnesses:

- Sandboxed execution (containers, VMs)
- File access controls
- Network restrictions
- Command allowlists/blocklists
- Human approval for dangerous ops

The Harness is the Security Boundary

The model can request **any tool call**. The harness decides what actually runs.

In production harnesses:

- Sandboxed execution (containers, VMs)
- File access controls
- Network restrictions
- Command allowlists/blocklists
- Human approval for dangerous ops

The Harness is the Security Boundary

The model can request **any tool call**. The harness decides what actually runs.

In production harnesses:

- Sandboxed execution (containers, VMs)
- File access controls
- Network restrictions
- Command allowlists/blocklists
- Human approval for dangerous ops

kodex: not that

kodex runs **everything unsandboxed**.

- shell=True, no sandbox
- Approval by command name only
- No file access restrictions
- Full network access

Fine for learning. Do not deploy.

What is kodex?

kodex is Swedish for "codex" - like IKEA furniture, you assemble it yourself.

An educational coding agent - like Claude Code or Gemini CLI.

- **Goal:** Teach agentic tool-use end to end
- **Stack:** Python + OpenAI SDK (compatible)
- **Size:** ~200 lines, two files
- **7 tools:** bash, read_file, write_file, edit, glob, grep, list_files

What is kodex?

kodex is Swedish for "codex" – like IKEA furniture, you assemble it yourself.

An educational coding agent – like Claude Code or Gemini CLI.

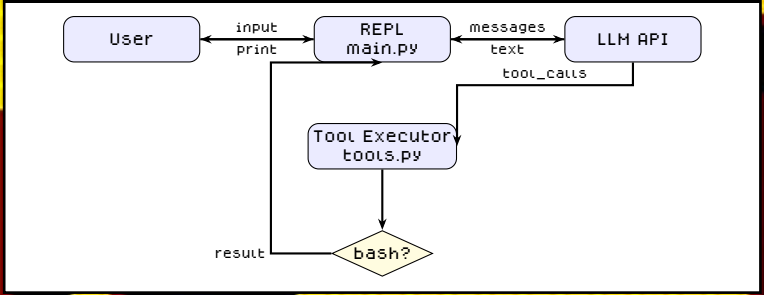
- **Goal:** Teach agentic tool-use end to end
- **Stack:** Python + OpenAI SDK (compatible)
- **Size:** ~200 lines, two files
- **7 tools:** bash, read_file, write_file, edit, glob, grep, list_files

Key idea

LLM + tools + loop = coding agent.

kodex

Architecture



Project Structure

```
kodex/
  pyproject.toml
  kodex/
    __init__.py
    __main__.py
    main.py        # REPL + agentic loop
    tools.py       # tool execution
    models.json    # endpoints + models
    prompt.txt     # system prompt
    tools.json     # tool schemas
```

Project Structure

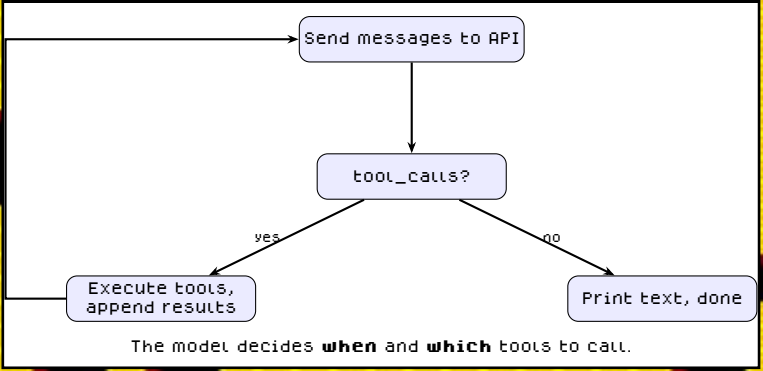
```
kodex/
  pyproject.toml
  kodex/
    __init__.py
    __main__.py
    main.py        # REPL + agentic loop
    tools.py       # tool execution
    models.json    # endpoints + models
    prompt.txt     # system prompt
    tools.json     # tool schemas
```

Config-driven

- **prompt.txt** – system prompt ($cwd placeholder)
- **tools.json** – tool schemas for the API
- **models.json** – endpoints + model aliases

Override via ~/.kodex/ – no source changes.

The Agentic Loop



The model decides **when** and **which** tools to call.

The Agentic Loop (main.py)

```python
while True:
    resp = client.chat.completions.create(
        model=model_id,
        messages=messages,
        tools=schemas,
    )
    msg = resp.choices[0].message
    messages.append(msg)

    if msg.content:
        print(msg.content)
    if not msg.tool_calls:
        break

    for tc in msg.tool_calls:
        result = execute(
            tc.function.name,
            tc.function.arguments,
        )
        messages.append({
            "role": "tool",
            "tool_call_id": tc.id,
            "content": result,
        })
```

The Agentic Loop (main.py)

```python
while True:
    resp = client.chat.completions.create(
        model=model_id,
        messages=messages,
        tools=schemas,
    )
    msg = resp.choices[0].message
    messages.append(msg)

    if msg.content:
        print(msg.content)
    if not msg.tool_calls:
        break

    for tc in msg.tool_calls:
        result = execute(
            tc.function.name,
            tc.function.arguments,
        )
        messages.append({
            "role": "tool",
            "tool_call_id": tc.id,
            "content": result,
        })
```

Tool Schema (tools.json)

```json
{ "type": "function",
  "function": {
    "name": "bash",
    "description": "Run a command.",
    "parameters": {
      "type": "object",
      "properties": {
        "command": {
          "type": "string"
        }
      },
      "required": ["command"]
} } }
```

Tool Execution (tools.py)

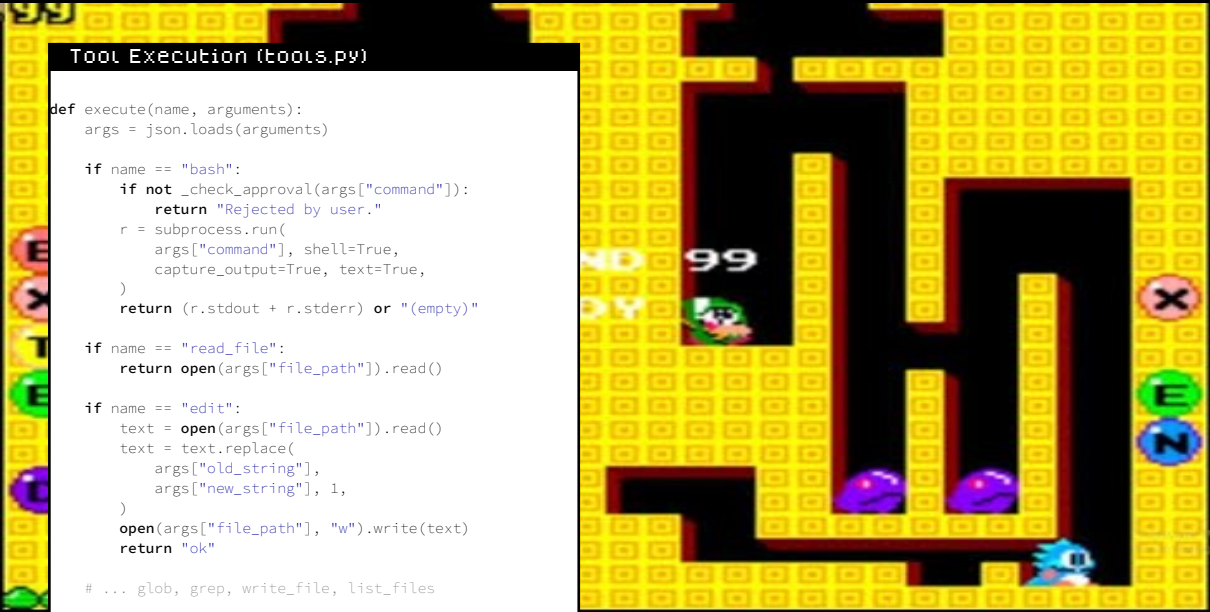
```python
def execute(name, arguments):
    args = json.loads(arguments)

    if name == "bash":
        if not _check_approval(args["command"]):
            return "Rejected by user."
        r = subprocess.run(
            args["command"], shell=True,
            capture_output=True, text=True,
        )
        return (r.stdout + r.stderr) or "(empty)"

    if name == "read_file":
        return open(args["file_path"]).read()

    if name == "edit":
        text = open(args["file_path"]).read()
        text = text.replace(
            args["old_string"],
            args["new_string"], 1,
        )
        open(args["file_path"], "w").write(text)
        return "ok"

    # ... glob, grep, write_file, list_files
```



Tool Execution (tools.py)

```python
def execute(name, arguments):
    args = json.loads(arguments)

    if name == "bash":
        if not _check_approval(args["command"]):
            return "Rejected by user."
        r = subprocess.run(
            args["command"], shell=True,
            capture_output=True, text=True,
        )
        return (r.stdout + r.stderr) or "(empty)"

    if name == "read_file":
        return open(args["file_path"]).read()

    if name == "edit":
        text = open(args["file_path"]).read()
        text = text.replace(
            args["old_string"],
            args["new_string"], 1,
        )
        open(args["file_path"], "w").write(text)
        return "ok"

    # ... glob, grep, write_file, list_files
```

Bash Approval

```python
_approved = set()

def _check_approval(cmd):
    name = cmd.split()[0]
    if name in _approved:
        return True
    choice = input(f"{cmd}? y/a/n: ")
    if choice == "a":
        _approved.add(name)
        return True
    return choice == "y"
```

kodex

Getting Started

```
mkdir -p ~/.kodex
echo '{"google":"..."}' > ~/.kodex/keys.json
uv run kodex
```

kodex

Getting Started

```
mkdir -p ~/.kodex
echo '{"google":"..."}' > ~/.kodex/keys.json
uv run kodex
```

Example Session

```
kodex | gemini-3.1-flash | google
cwd: /home/user/myproject

> write me a hello world in python
--- write_file ---
Created hello.py

--- bash ---
  Command: python hello.py
  Allow? (y)es / (a)lways / (n)o: y
Hello, world!
```

kodex

Extend
New tool: schema in tools.json + branch in execute()
New model: entry in models.json
New prompt: edit prompt.txt
New endpoint: entry in models.json
ROUND 99
READY
EXTEND
EXTEN

kodex

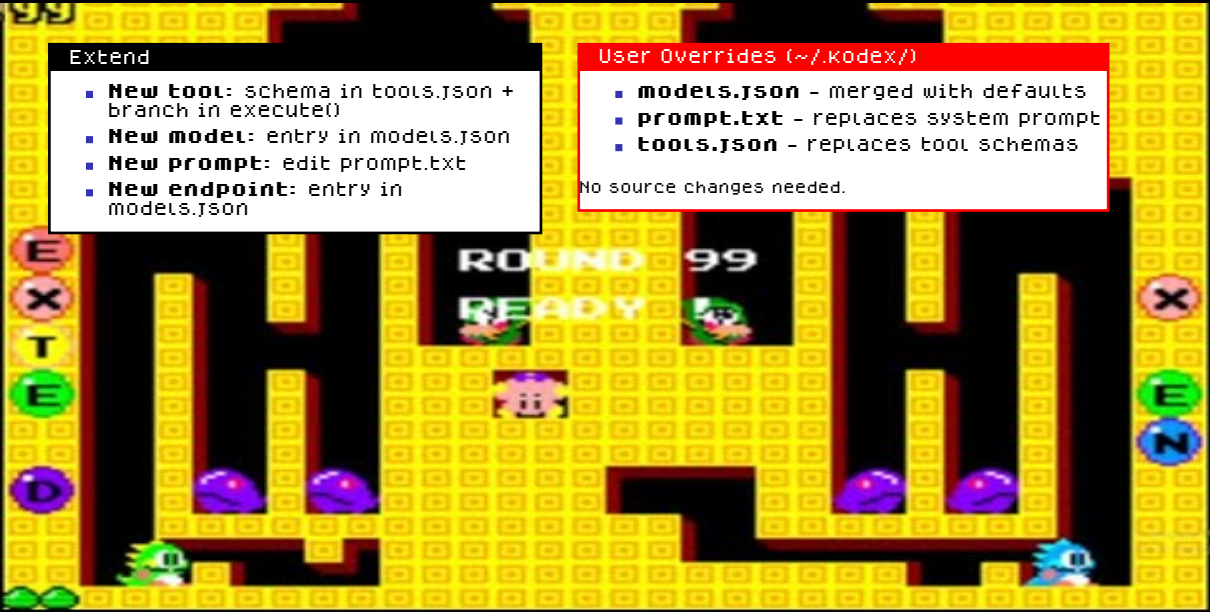
Extend

- **New tool:** schema in tools.json + branch in execute()
- **New model:** entry in models.json
- **New prompt:** edit prompt.txt
- **New endpoint:** entry in models.json

User Overrides (~/.kodex/)

- **models.json** – merged with defaults
- **prompt.txt** – replaces system prompt
- **tools.json** – replaces tool schemas

No source changes needed.



Takeaways

1. **A coding agent is just a loop.** Send messages, execute tool calls, repeat.
2. **The model decides what to do.** You provide tools and schemas; it picks which to call and when.
3. **OpenAI SDK is the common interface.** Gemini, OpenAI, open-source providers all support it.
4. **Approval gates are trivial but essential.** One function guards shell access.
5. **Everything is a config file.** Swap prompts, tools, models, endpoints without touching code.

Takeaways

1. **A coding agent is just a loop.** Send messages, execute tool calls, repeat.
2. **The model decides what to do.** You provide tools and schemas; it picks which to call and when.
3. **OpenAI SDK is the common interface.** Gemini, OpenAI, open-source providers all support it.
4. **Approval gates are trivial but essential.** One function guards shell access.
5. **Everything is a config file.** Swap prompts, tools, models, endpoints without touching code.

github.com/cynthia/kodex

https://github.com/cynthia/kodex

References

[1] Alec Radford et al. "Improving Language Understanding by Generative Pre-Training". In: (2018). OpenAI.

[2] Ashish Vaswani et al. "Attention is All You Need". In: Advances in Neural Information Processing Systems 30 (2017).

[3] Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units". In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (2016), pp. 1715-1725.

[4] Tom Brown et al. "Language Models are Few-Shot Learners". In: Advances in Neural Information Processing Systems 33 (2020), pp. 1877-1901.

[5] Long Ouyang et al. "Training Language Models to Follow Instructions with Human Feedback". In: Advances in Neural Information Processing Systems 35 (2022), pp. 27730-27744.